Bloom Filter-Based MPSI

Weekly Progress Meeting 26 Feb 2026

Andra Alăzăroaie

Supervisor: Lilika Markatou

Daily Supervisor: Tjitske Koster

26-02-2026

Bibliography

[1] Jelle Vos, Jorrit van Assen, Tjitske Koster, Evangelia Anna Markatou, and Zekeriya Erkin. On the insecurity of bloom filter-based private set intersections. Cryptology ePrint Archive, 2024.

[3] Ou Ruan, Changwang Yan, Jing Zhou, and Chaohao Ai. A practical multiparty private set intersection protocol based on bloom filters for unbalanced scenarios. Applied Sciences, 13(24):13215, 2023.

[6] Vladimir Kolesnikov, Naor Matania, Benny Pinkas, Mike Rosulek, and Ni Trieu. Practical multi-party private set intersection from symmetric-key techniques.In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pages 1257–1272, 2017.

[7] Alireza Kavousi, Javad Mohajeri, and Mahmoud Salmasizadeh. Efficient scalable multi-party private set intersection using oblivious prf. In International Workshop on Security and Trust Management, pages 81–99. Springer, 2021.

[8] Florian Kerschbaum. Outsourced private set intersection using homomorphic encryption. In Proceedings of the 7th ACM Symposium on Information, Computer and Communications Security, pages 85–86, 2012.

[9] Casacuberta, Sílvia, Julia Hesse, and Anja Lehmann. "SoK: oblivious pseudorandom functions." In 2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P), pp. 625-646. IEEE, 2022.

[10] Dong, Changyu, Liqun Chen, and Zikai Wen. "When private set intersection meets big data: an efficient and scalable protocol." In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, pp. 789-800. 2013.

[11] Falzon, Francesca, and Evangelia Anna Markatou. "Re-visiting authorized private set intersection: A new privacy-preserving variant and two protocols." *Proceedings on Privacy Enhancing Technologies* (2025).

Progress

From the plan last week:

- Run the experiments on the OPRF mitigation
- Implement the Garbled BF mitigation
- Read more and do some thinking on the Authorized MPSI mitigation

Progress

OPRFs:

Pre-processing stage:

- Clients generate a key k_{OPRF}, they distribute it in a star topology
- Original protocol: each client hashes their element using the k hash functions to get the indices in the BF
- Mitigation: each client computes the indices by using their k_{OPRF}
  - For element x: $H_1(H(x)^{k_{OPRF}}\ mod\ p), H_2(H(x)^{k_{OPRF}} mod\ p), \dots, H_k(H(x)^{k_{OPRF}} mod\ p)$
  - $\gcd(k_{OPRF}, \mathrm{p} - 1) = 1$, so $H(x)^{k_{OPRF}}\ mod\ p$ will be unique for $H(x)$
  - The hash function $H(x)$ needs to generate uniformly random elements, something like SHA256 should work?
  - $H_1, H_2, \dots, H_k$ are the previous hash functions mapping elements to bins

Progress

OPRFs:

Online stage:

- Original protocol: server computes the k hashes for each of its elements to get indices and collect the encrypted values at those indices from the clients' encrypted BFs
- Mitigation: server has to engage in an OPRF protocol in order to get the indices
  - One OPRF is sufficient since all clients have the same k_{OPRF}

```
// Server S initiates a request:
Request(M) → t, B
    t ←$ Z_q
    B ← {a^t}_{a∈M}
    return t, B

// Client C applies PRF:
Eval(B, k) → C
    return {b^k}_{b∈B}

// Server S recovers elements:
Recover(C, t) → D
    return {c^(1/t)}_{c∈C}
```

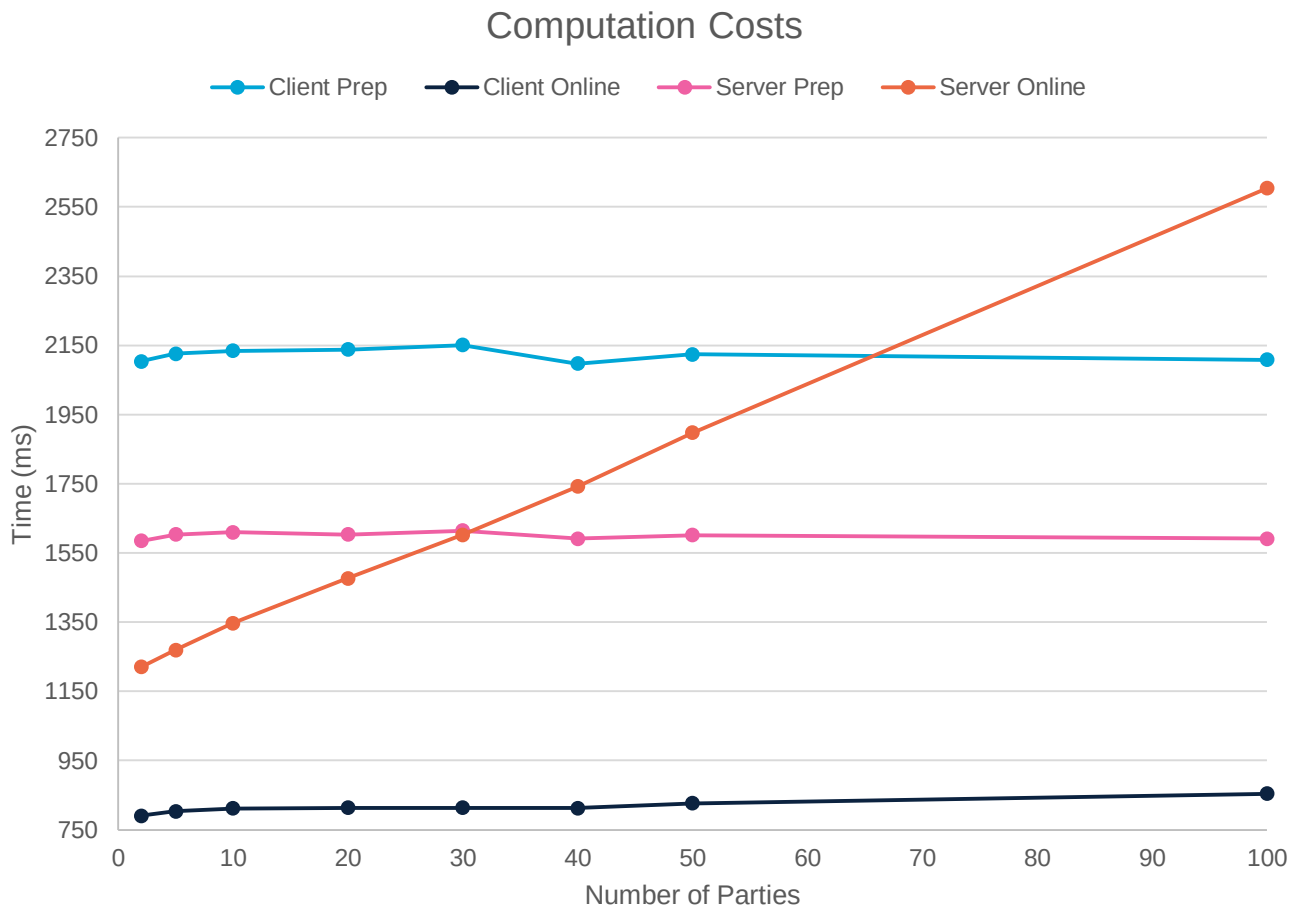
Figure 8: OPRF Protocol

OPRF

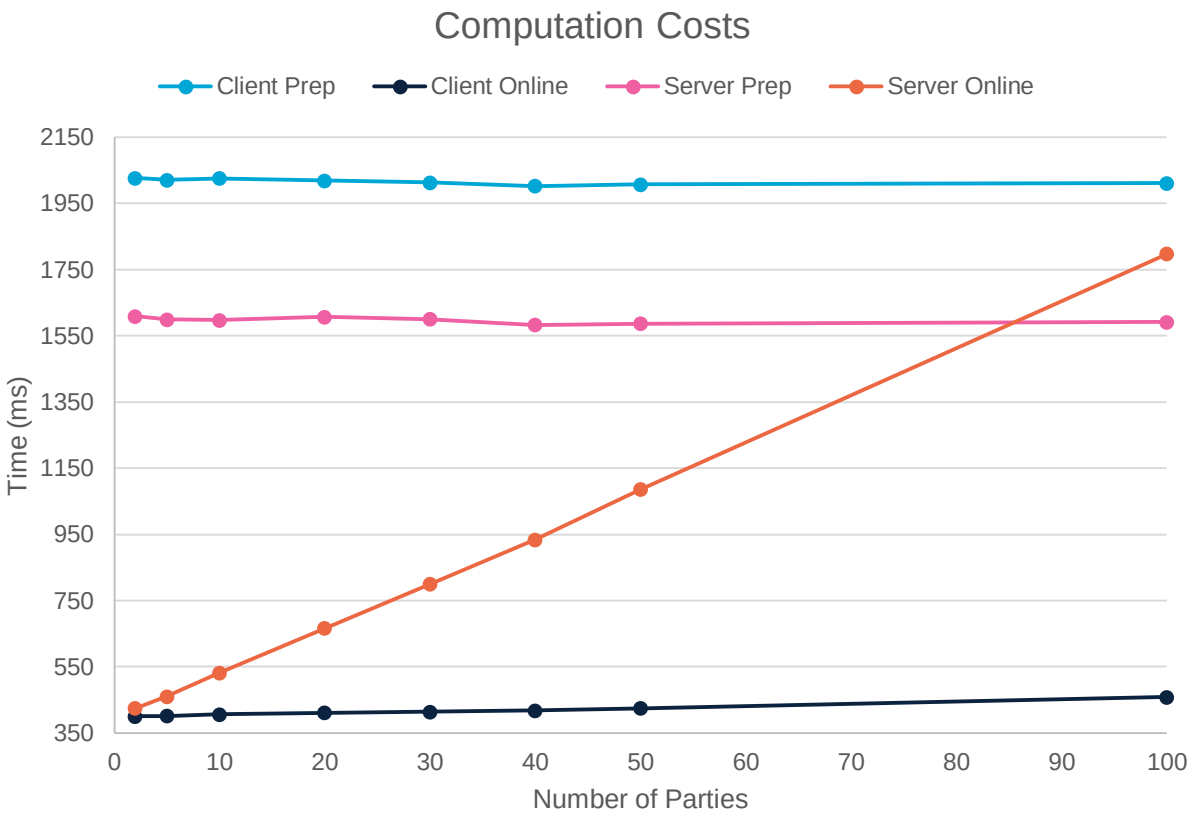
Ruan et al. [3] results, **OPRF Mitigation (RQ3)**:

Computation time (in ms) per party for a run of the protocol involving various number of parties (including the server)

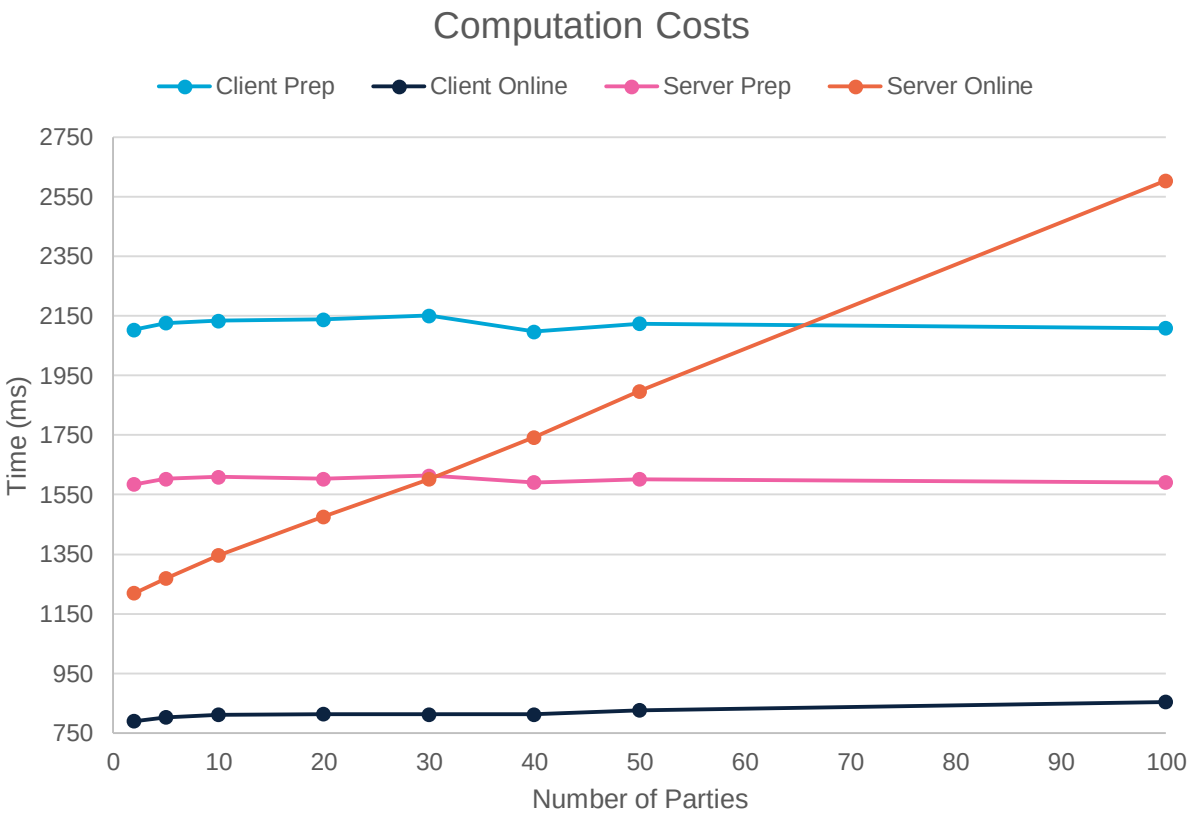
Set size clients: 256 elements
Set size server: 1024 elements
False positive rate: 2^{-7}



Baseline



OPRF

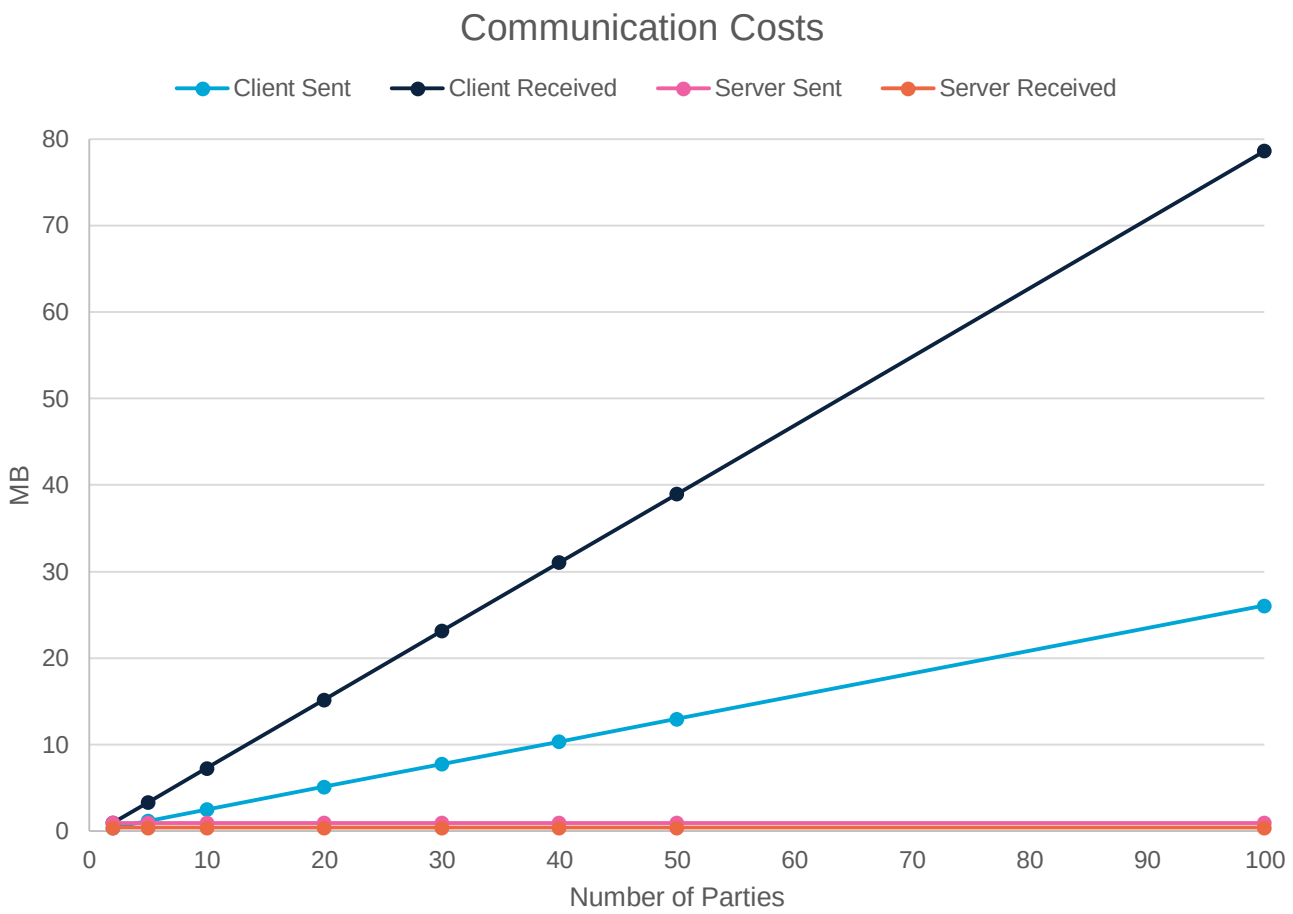


OPRF

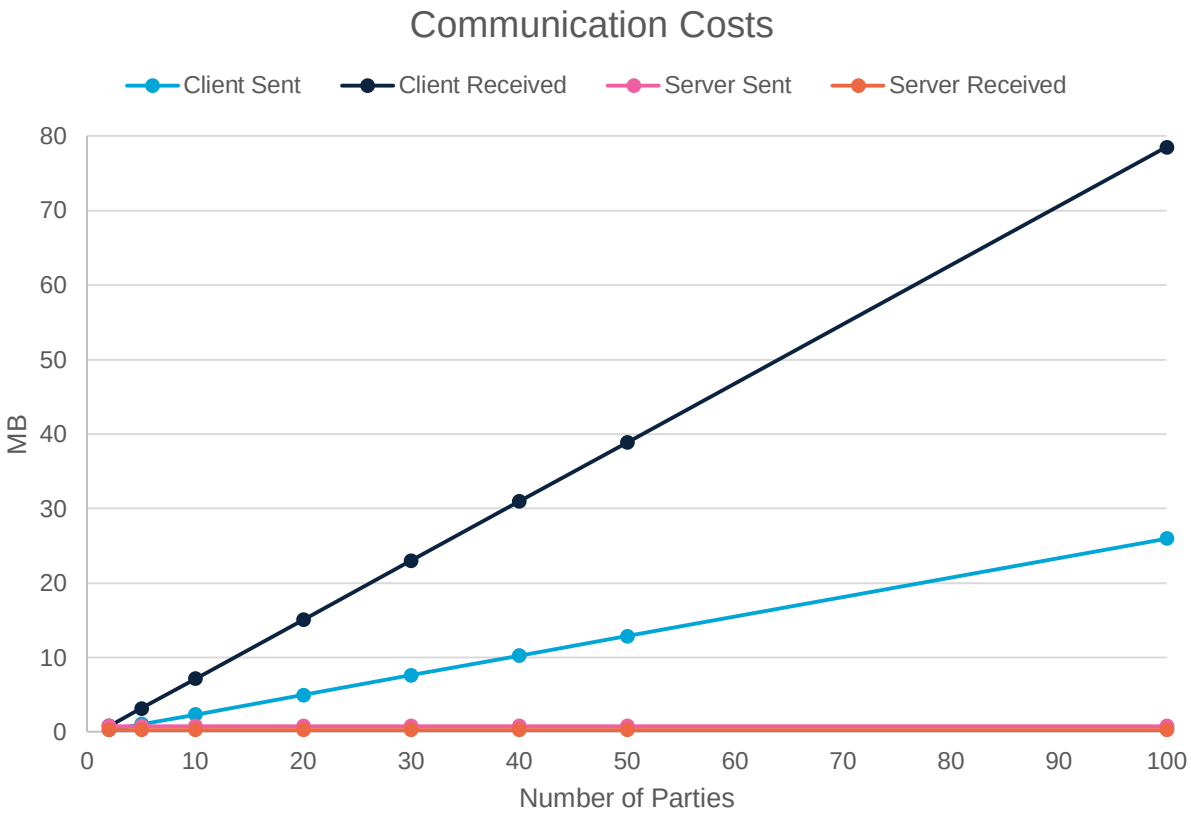
Ruan et al. [3] results, **OPRF Mitigation (RQ3)**:

Communication size (in MB) per party for a run of the protocol involving various number of parties (including the server)

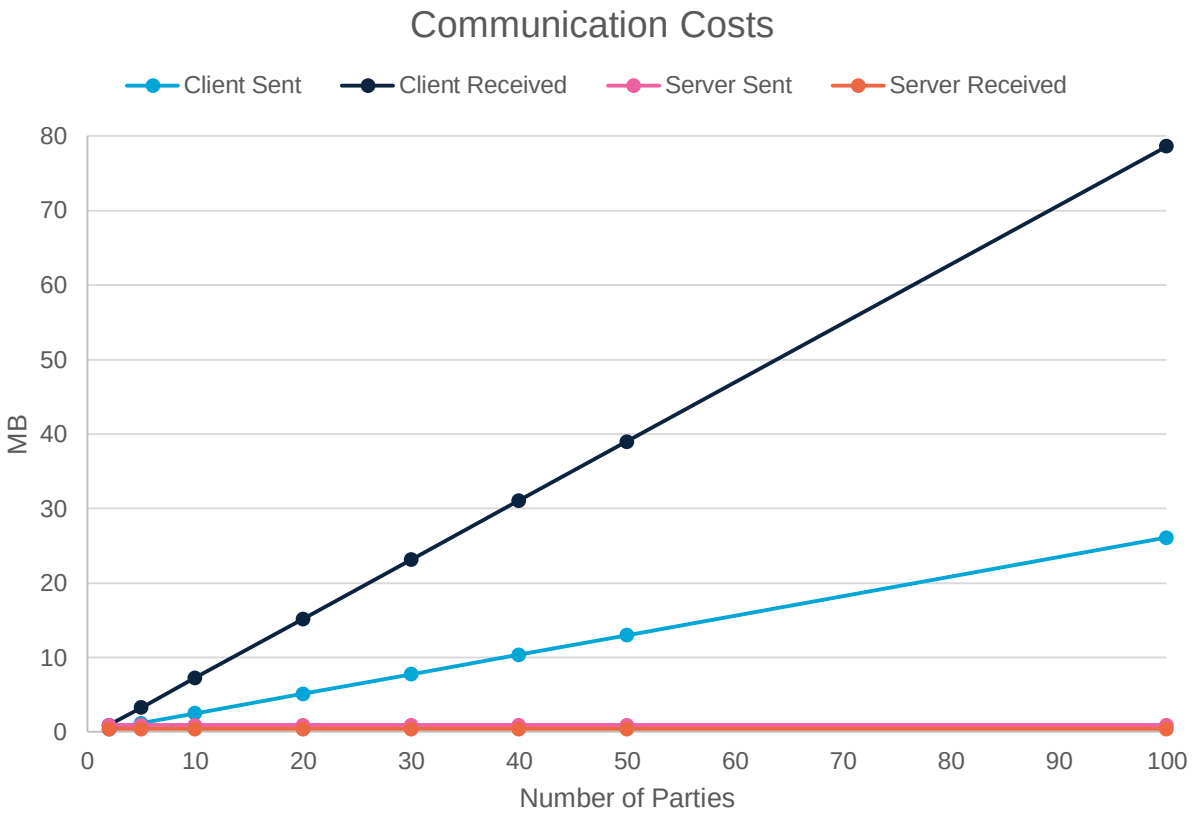
Set size clients: 256 elements
Set size server: 1024 elements
False positive rate: 2^{-7}



Baseline



OPRF



OPRF

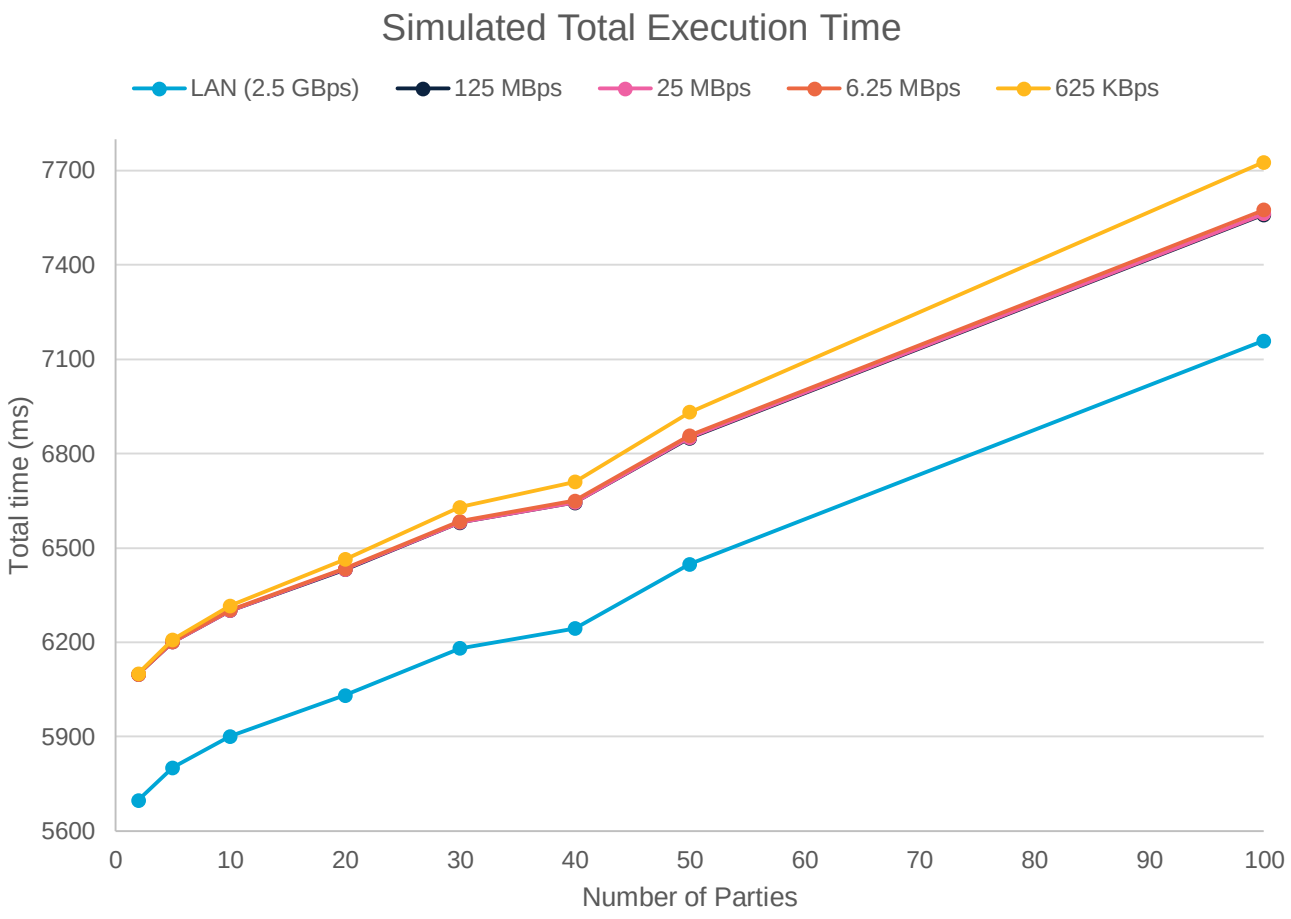
Ruan et al. [3] results, **OPRF Mitigation (RQ3)**:

Total time (in ms) for a run of the protocol involving various number of parties (including the server)

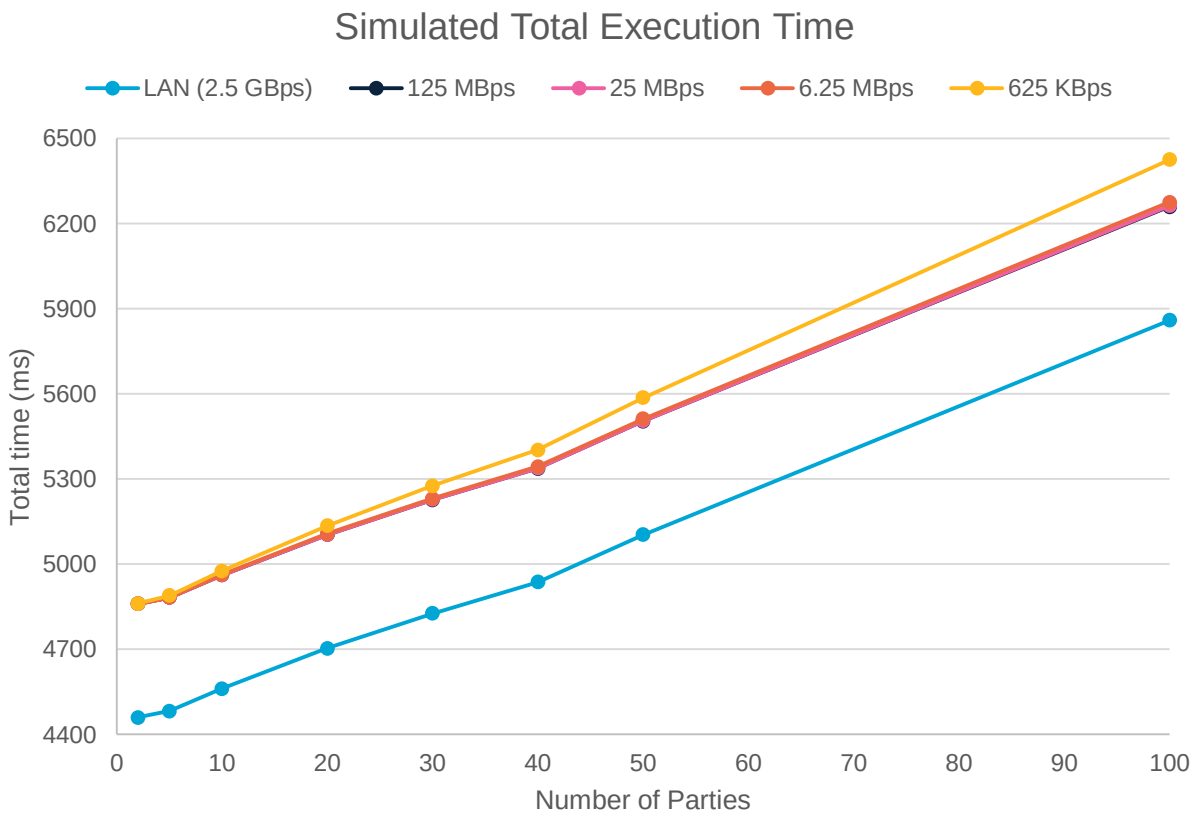
Set size clients: 256 elements
Set size server: 1024 elements
False positive rate: 2^{-7}

Total time calculated as:
Latency + Communication time + Computation Time

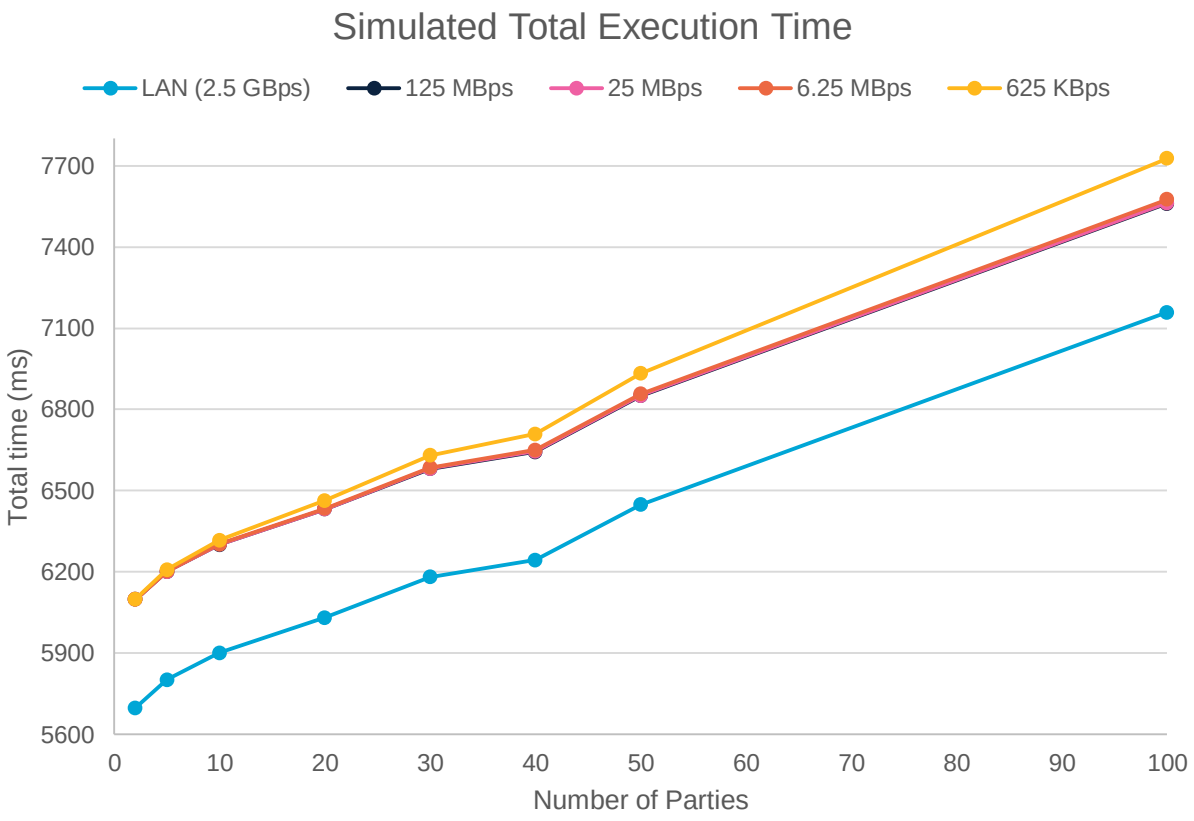
- **Latency:** *number of messages * hardcoded network latency*
- **Communication time:** *Bytes sent * hardcoded bandwidth*
- **Computation time:** plotted before



Baseline



OPRF

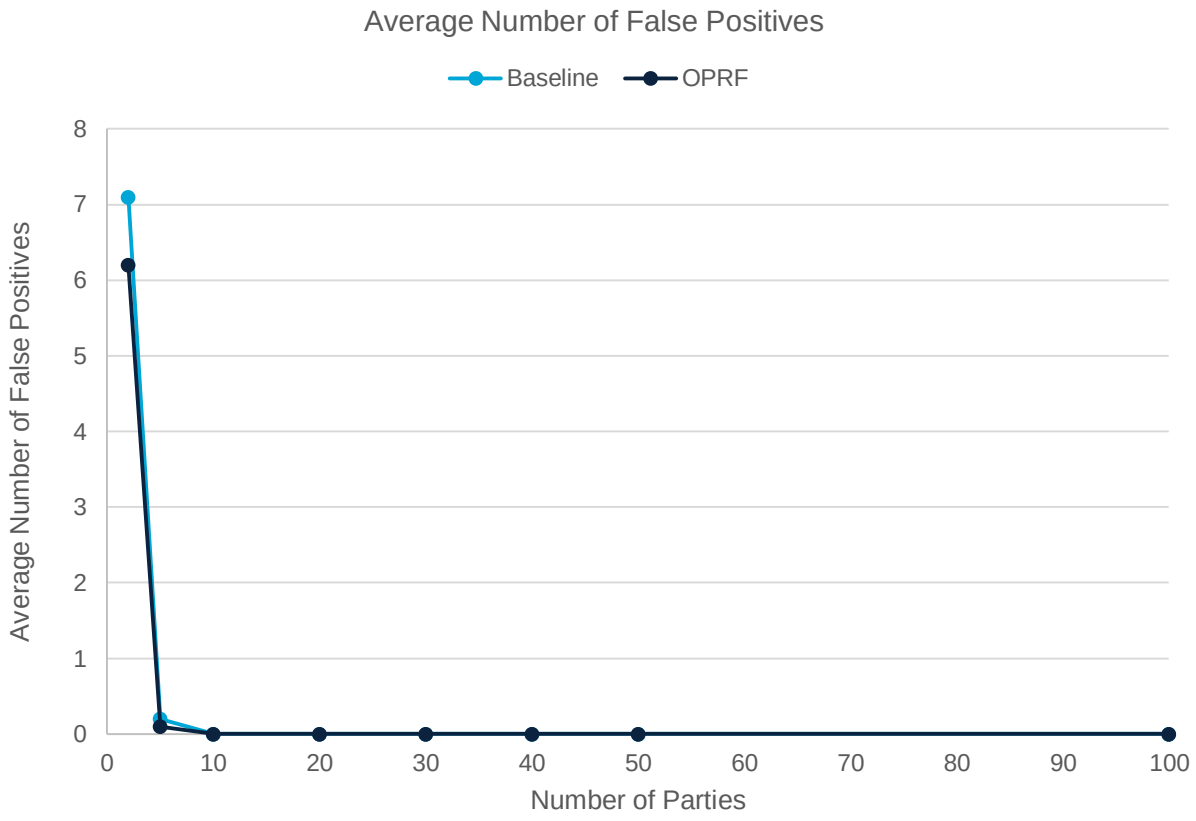


OPRF

Ruan et al. [3] results, **OPRF Mitigation (RQ3)**:

(Average) Number of False Positives for a run of the protocol involving various number of parties (including the server) **for the baseline vs mitigation**

Set size clients: 256 elements
Set size server: 1024 elements



Progress

GBFs [10]:

- Repeating hash values makes the algorithm fail
- Take k=3, m=5:
  - $h_1(x) = 1: emptySlot = 1$
  - $h_2(x) = 4: GBF[4] = s_1$
  - $h_3(x) = 1: GBF[1] = s_2$
  - $GBF[emptySlot] = GBF[1] = 4 \oplus s1 \oplus s_2$

Algorithm 1: $BuildGBF(S, n, m, k, H, \lambda)$

```
input  : A set S, n, m, k, λ, H = {h_0, ...h_{k-1}}
output: An (m, n, k, H, λ)-garbled Bloom filter GBF_S
GBF_S= new m-element array of bit strings;
for i= 0 to m − 1 do
    GBF_S[i]=NULL;                    // NULL is the special symbol that means ''no value''
end
for each x ∈ S do
    emptySlot = −1, finalShare= x;
    for i=0 to k-1 do
        j = h_i(x);                   // get an index by hashing the element
        if GBF_S[j]==NULL then
            if emptySlot ==−1 then
                emptySlot=j;          // reserve this location for finalShare
            else
                GBF_S[j] ←r {0,1}^λ;  // generate a new share
                finalShare=finalShare⊕GBF_S[j];
            end
        else
            finalShare=finalShare⊕GBF_S[j];   // reuse a share
        end
    end
    GBF_S[emptySlot]=finalShare;      // store the last share
end
for i= 0 to m − 1 do
    if GBF_S[i]==NULL then
        GBF_S[i] ←r {0,1}^λ;
    end
end
```

Progress

GBFs [10]:

- $GBF[1] = 4 \oplus s1 \oplus s_2$,
- $GBF[4] = s_1$,
- The rest are random
- Recovering x would do:
  - $GBF[1] \oplus GBF[4] \oplus GBF[1] = (4$

Algorithm 2: $QueryGBF(GBF_S, x, k, H)$

input : A gabled Bloom filter GBF_S, an element x, k, $H = \{h_0, ...h_{k-1}\}$
output: True if $x \in S$, False otherwise

```
recovered = {0}^λ;
for i=0 to k-1 do
    j = h_i(x);
    recovered = recovered ⊕ GBF_S[j];
end
if recovered == x then
    return True;
else
    return False;
end
```

Progress

GBFs [10]:

- A solution is to make sure that the indices generated by the hash functions are unique

Algorithm 1: $BuildGBF(S, n, m, k, H, \lambda)$

```
input  : A set S, n, m, k, λ, H = {h_0, ...h_{k-1}}
output: An (m, n, k, H, λ)-garbled Bloom filter GBF_S
GBF_S= new m-element array of bit strings;
for i= 0 to m − 1 do
    GBF_S[i]=NULL;                          // NULL is the special symbol that means ''no value''
end
for each x ∈ S do
    emptySlot = −1, finalShare= x;
    for i=0 to k-1 do
        j = h_i(x);                         // get an index by hashing the element
        if GBF_S[j]==NULL then
            if emptySlot ==−1 then
                emptySlot=j;                // reserve this location for finalShare
            else
                GBF_S[j] ←r {0,1}^λ;        // generate a new share
                finalShare=finalShare⊕GBF_S[j];
            end
        else
            finalShare=finalShare⊕GBF_S[j]; // reuse a share
        end
    end
    GBF_S[emptySlot]=finalShare;            // store the last share
end
for i= 0 to m − 1 do
    if GBF_S[i]==NULL then
        GBF_S[i] ←r {0,1}^λ;
    end
end
```

Progress

GBFs [10] adaptation to **Ruan et al. [6]** :

- Instead of the XOR of shares giving x : work in the same (multiplicative) group used by ElGamal, the shares are group elements

Algorithm 1: $BuildGBF(S, n, m, k, H, \lambda)$

```
input  : A set S, n, m, k, λ, H = {h_0, ...h_{k-1}}
output: An (m, n, k, H, λ)-garbled Bloom filter GBF_S
GBF_S= new m-element array of bit strings;
for i= 0 to m − 1 do
    GBF_S[i]=NULL;                          // NULL is the special symbol that means ''no value''
end
for each x ∈ S do
    emptySlot = −1, finalShare= x;
    for i=0 to k-1 do
        j = h_i(x);                         // get an index by hashing the element
        if GBF_S[j]==NULL then
            if emptySlot ==−1 then
                emptySlot=j;                // reserve this location for finalShare
            else
                GBF_S[j] ←r {0,1}^λ;        // generate a new share
                finalShare=finalShare⊕GBF_S[j];
            end
        else
            finalShare=finalShare⊕GBF_S[j]; // reuse a share
        end
    end
    GBF_S[emptySlot]=finalShare;            // store the last share
end
for i= 0 to m − 1 do
    if GBF_S[i]==NULL then
        GBF_S[i] ←r {0,1}^λ;
    end
end
```

Progress

GBFs [10] adaptation to **Ruan et al. [6]** :

- Original Protocol:
  - The server multiplies the encrypted bins (encrypted 1's in the original protocol) with a blinded version of its element
  - Adds to the intersection if the decryption is the blinded element

Algorithm 1: $BuildGBF(S, n, m, k, H, \lambda)$

```
input  : A set S, n, m, k, λ, H = {h_0, ...h_{k-1}}
output: An (m, n, k, H, λ)-garbled Bloom filter GBF_S
GBF_S= new m-element array of bit strings;
for i= 0 to m − 1 do
    GBF_S[i]=NULL;                          // NULL is the special symbol that means ''no value''
end
for each x ∈ S do
    emptySlot = −1, finalShare= x;
    for i=0 to k-1 do
        j = h_i(x);                         // get an index by hashing the element
        if GBF_S[j]==NULL then
            if emptySlot ==−1 then
                emptySlot=j;                // reserve this location for finalShare
            else
                GBF_S[j] ←r {0,1}^λ;        // generate a new share
                finalShare=finalShare⊕GBF_S[j];
            end
        else
            finalShare=finalShare⊕GBF_S[j]; // reuse a share
        end
    end
    GBF_S[emptySlot]=finalShare;            // store the last share
end
for i= 0 to m − 1 do
    if GBF_S[i]==NULL then
        GBF_S[i] ←r {0,1}^λ;
    end
end
```

Progress

GBFs [10] adaptation to **Ruan et al. [6]** :

- GBF mitigation:
  - The server multiplies the encrypted bins (shares of the element) with a blinded version of its element
  - Adds to the intersection if the decryption is the blinded element multiplied with the element itself

Algorithm 1: $BuildGBF(S, n, m, k, H, \lambda)$

```
input  : A set S, n, m, k, λ, H = {h_0, ...h_{k-1}}
output: An (m, n, k, H, λ)-garbled Bloom filter GBF_S
GBF_S= new m-element array of bit strings;
for i= 0 to m − 1 do
    GBF_S[i]=NULL;                          // NULL is the special symbol that means ''no value''
end
for each x ∈ S do
    emptySlot = −1, finalShare= x;
    for i=0 to k-1 do
        j = h_i(x);                         // get an index by hashing the element
        if GBF_S[j]==NULL then
            if emptySlot ==−1 then
                emptySlot=j;                // reserve this location for finalShare
            else
                GBF_S[j] ←r {0,1}^λ;        // generate a new share
                finalShare=finalShare⊕GBF_S[j];
            end
        else
            finalShare=finalShare⊕GBF_S[j]; // reuse a share
        end
    end
    GBF_S[emptySlot]=finalShare;            // store the last share
end
for i= 0 to m − 1 do
    if GBF_S[i]==NULL then
        GBF_S[i] ←r {0,1}^λ;
    end
end
```

Progress

GBFs [10] adaptation to **Ruan et al. [6]** :

- GBF mitigation:
  - The server multiplies the encrypted bins (shares of 1) with a blinded version of its element
  - Adds to the intersection if the decryption is the blinded element

Algorithm 1: $BuildGBF(S, n, m, k, H, \lambda)$

```
input  : A set S, n, m, k, λ, H = {h_0, ...h_{k-1}}
output: An (m, n, k, H, λ)-garbled Bloom filter GBF_S
GBF_S= new m-element array of bit strings;
for i= 0 to m − 1 do
    GBF_S[i]=NULL;                    // NULL is the special symbol that means ''no value''
end
for each x ∈ S do
    emptySlot = −1, finalShare= x;
    for i=0 to k-1 do
        j = h_i(x);                   // get an index by hashing the element
        if GBF_S[j]==NULL then
            if emptySlot ==−1 then
                emptySlot=j;          // reserve this location for finalShare
            else
                GBF_S[j] ←r {0,1}^λ;  // generate a new share
                finalShare=finalShare⊕GBF_S[j];
            end
        else
            finalShare=finalShare⊕GBF_S[j];   // reuse a share
        end
    end
    GBF_S[emptySlot]=finalShare;      // store the last share
end
for i= 0 to m − 1 do
    if GBF_S[i]==NULL then
        GBF_S[i] ←r {0,1}^λ;
    end
end
```

Next Week

- Finalize and benchmark GBF
- Do some writing on the mitigations so far
- Continue with the Authorized MPSI mitigation (read some references of [11])

Thank you!

26-02-2026